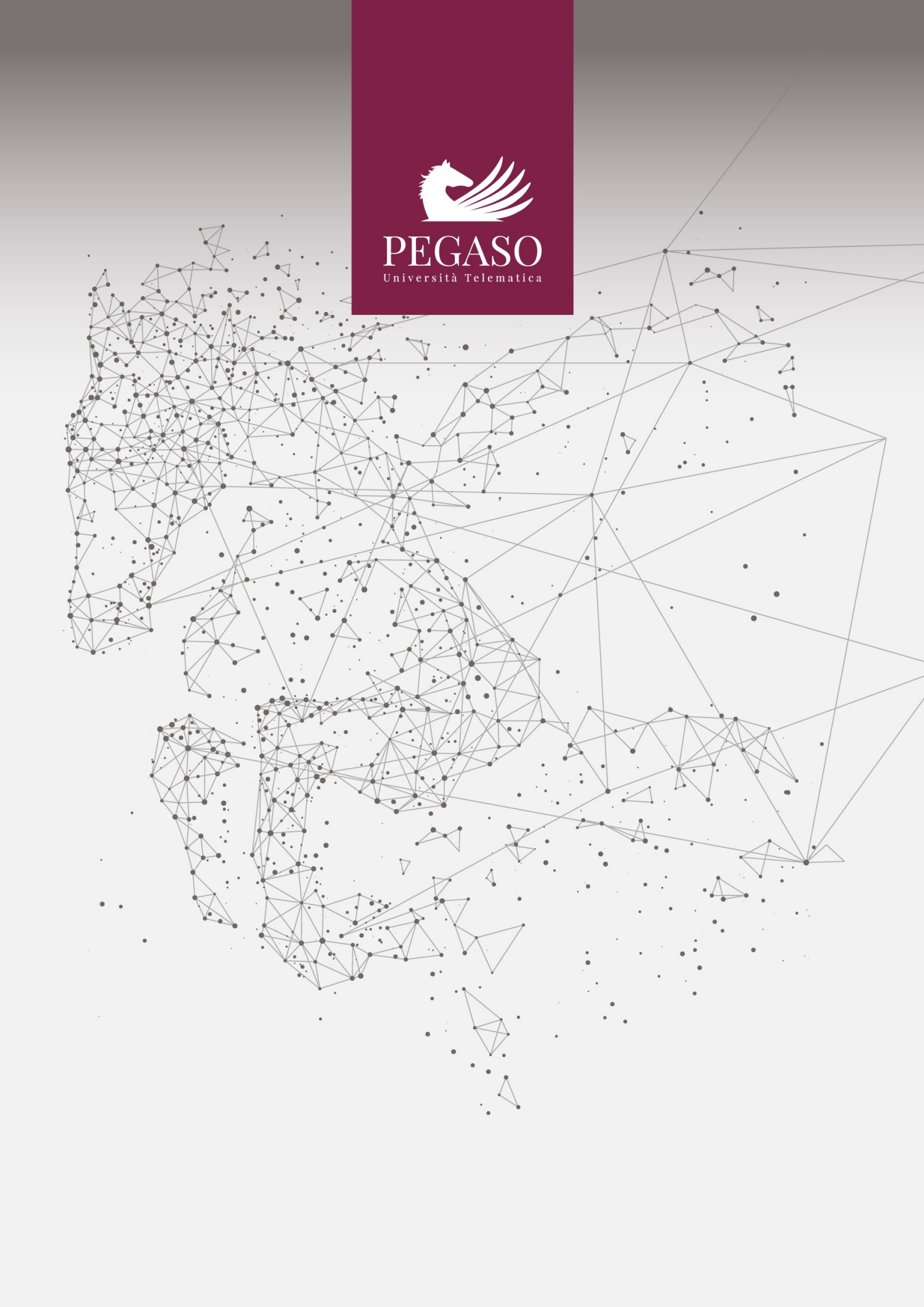

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. ANALISI DEL PROBLEMA....4
3. PROGETTAZIONE (OBJECT ORIENTED DESIGN) DELLA SOLUZIONE....6
4. FORMALIZZAZIONE DELLA SOLUZIONE IN PYTHON....11
5. SIMULAZIONE DI ALCUNE OPERAZIONI....21
BIBLIOGRAFIA....25

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, viene presentata un'esercitazione di approfondimento sulla modellazione e successiva formalizzazione di un sistema software in linguaggio Python seguendo la metodologia OOP (Object Oriented Programming). Nel primo paragrafo viene illustrato il contesto e l'analisi del sistema software richiesto da una piccola agenzia immobiliare. Dopo aver individuato i dati da gestire e le attività da automatizzare, vengono modellate le classi con i relativi attributi e metodi. Nel secondo paragrafo viene descritta graficamente la struttura del sistema in termini di classi e legami tra di esse. Nel terzo paragrafo viene illustrata la formalizzazione delle classi e della funzione di simulazione *main()* in linguaggio Python. Infine, vengono illustrati i risultati di alcune esecuzioni del programma al fine di simulare le operazioni realizzate.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Analisi del problema

In questa esercitazione verrà sviluppata una piccola applicazione per permettere ad una agenzia immobiliare di svolgere le proprie attività di vendita delle case che riceve in gestione. Le tipiche attività che l'applicazione deve rendere disponibili per supportare l'operatore nelle attività quotidiane sono:

1) l'aggiunta di una nuova casa in gestione;
2) la cancellazione di una casa dopo la vendita;
3) la stampa a video delle informazioni delle case in gestione;
4) la ricerca delle case all'interno di un range di prezzo da proporre al cliente;
5) il mantenimento delle schede con i dati delle case in gestione su un supporto di memoria permanente.

In particolare, per ogni casa data in gestione all'agenzia immobiliare bisogna gestire le seguenti informazioni:

1) un codice univoco alfanumerico che identifica la casa;
2) il prezzo di vendita;
3) il numero di vani;
4) i mq della casa;
5) se possiede un garage;
6) se possiede un ascensore;
7) se si trova in un condominio;
8) l'indirizzo completo (via con numero, cap, comune e provincia);
9) informazioni sul proprietario:
    a) nominativo;
    b) numero telefono fisso;
    c) numero cellulare.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

A titolo di esempio, di seguito viene proposto un esempio di una possibile scheda per una casa data in gestione all'agenzia.

```
Codice: s001
      Prezzo: 156000.00 €
      N. vani: 6
      Metri quadri: 145.0
      Ascensore: N
      Condominio: N
      Garage: S
      Indirizzo: Via Dante Alighieri 34
      CAP: 50100
      Comune: Capannori
      Provincia: LU
      -PROPRIETARIO-

      Alberto, Rossi, 0583987654, 338934567
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Progettazione (Object Oriented Design) della soluzione

La metodologia OOD consiste nel descrivere l'architettura di un sistema software in termini di classi (con i relativi attributi e metodi) e legami esistenti tra di esse. Dall'analisi del problema possiamo facilmente individuare le seguenti tre classi:

1) Casa - oggetto/entità che dobbiamo gestire;
2) Persona - oggetto/entità proprietario di una casa;
3) Collezione - oggetto/entità che colleziona tutte le case in gestione all'agenzia.

Inoltre, per supportare tutte le attività quotidiane di un operatore, il programma ha bisogno anche di:

- un insieme di funzioni per la memorizzazione su file della collezione per non perdere le schede informative delle case;
- un insieme di funzioni ausiliari di supporto per rendere più leggibile e manutenibile il sistema software.

Nel seguito, per comodità di trattazione, la lista dei parametri dei metodi che descriveremo, verrà sempre omesso il parametro "self".

Cominciamo con la classe ***Casa*** e descriviamo brevemente i suoi attributi:

- __codice codice alfanumerico identificativo;
- __prezzo prezzo della casa;
- __n_vani il numero dei vani;
- __mq i metri quadri;
- __ascensore presenza dell'ascensore [S/N];
- __condominio se la casa è in un condominio [S/N];
- __garage se possiede un garage [S/N];
- __indirizzo indirizzo che contiene anche il n. civico;
- __cap codice postale di cinque cifre;
- __comune il nome del comune;
- __provincia la sigla della provincia;
- __proprietario le informazioni del proprietario.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

A seguire si definiscono i metodi che modellano il «comportamento» della classe:

- i metodi *setter* che memorizzano/aggiornano il valore degli attributi:
    - set__codice(vcodice);
    - set__prezzo(vprezzo);
    - set__n_vani(vnvani);
    - set__mq(vmq);
    - set__ascensore(vascensore);
    - set__condominio(vcondominio);
    - set__garage(vgarage);
    - set__indirizzo(vindirizzo);
    - set__cap(vcap);
    - set__comune(vcomune);
    - set__provincia(vprovincia);
- i metodi *getter* che restituiscono/leggono il valore degli attributi:
    - get__codice();
    - get__prezzo();
    - get__n_vani();
    - get__mq();
    - get__ascensore();
    - get__condominio();
    - get__garage();
    - get__indirizzo();
    - get__cap();
    - get__comune();
    - get__provincia();
- i metodi di utilità generale e il costruttore della classe:
    - *__str__()* - riscrittura del metodo privato standard che restituisce una stringa per la rappresentazione a video degli oggetti della classe Casa;
    - *get__all_info()* - restituisce una stringa formattata per la scrittura su file di testo CSV delle informazioni degli oggetti della classe Casa;
    - *_init__(codice, prezzo, n_vani, mq, ascensore, condominio, garage, indi, cap, comune, pv, prop)* - è il metodo costruttore degli oggetti:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- riceve come parametri i valori di tutti gli attributi e li memorizza nelle variabili;
- restituisce un oggetto/istanza della classe Casa.

Allo stesso modo, partendo dall'analisi delle informazioni specifici del proprietario di una casa, si procede con la modellazione/descrizione della classe ***Persona*** partendo sempre dai suoi attributi:

- *__nome* il nome del proprietario;
- *__cognome* il cognome del proprietario;
- *__tel* il numero di telefono fisso del proprietario;
- *__cell* il numero di cellulare del proprietario.

A seguire il «comportamento» della classe e, quindi, i suoi metodi:

- metodi *setter* che memorizzano/aggiornano il valore degli attributi:
  - *set__nome(vnome);*
  - *set__cognomen(vcognome);*
  - *set__tel(vtel);*
  - *set__cell(vcell);*
- metodi *getter* che restituiscono/leggono il valore degli attributi:
  - *get__nome;*
  - *get__cognome;*
  - *get__tel;*
  - *get__cell.*

Infine, si riportano i metodi di utilità generale e il costruttore della classe:

- *__str__()* - riscrittura del metodo privato standard che restituisce una stringa per la rappresentazione a video degli oggetti della classe *Persona;*
- *get__all_info()* - restituisce una stringa formattata per la scrittura su file di testo CSV delle informazioni degli oggetti della classe Persona;
- *_init__(nome, cognome, tel, cell)* - è il metodo costruttore degli oggetti:
  - riceve come parametri i valori di tutti gli attributi e li memorizza nelle variabili;
  - restituisce un oggetto/istanza della classe Persona.

Per quanto riguarda la classe ***Collezione*** non vi è la necessità di modellare una nuova classe poiché tra i tipi di dati/classi native del linguaggio Python esiste la *<class 'dict'>* che modella proprio una collezione di dati/oggetti nella forma di una coppia: chiave (*key*) - valore (*value*).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per realizzare una collezione di Case, quindi, verrà impiegato un oggetto di tipo *<class 'dict'>* che verrà utilizzato nel seguente modo:

- chiave (*key*) valorizzata con il codice della *Casa;*
- valore (*value*) valorizzato con l'oggetto *Casa* corrispondente.

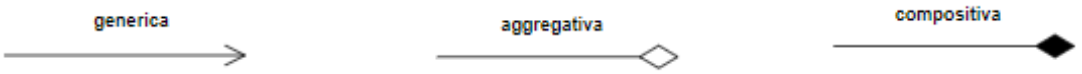
A questo punto, sono stati descritti quelli che saranno gli elementi strutturali del sistema. Per completare la modellazione, però, è necessario descrivere anche come le classi sono legate/interagiscono fra di loro. A tal proposito, si ritiene indicato fare prima alcuni brevi richiami di UML.

Il linguaggio di modellazione UML (Unified Modeling Language) altro non è che un formalismo grafico che consente di definire, mediante vari tipi di diagrammi, un sistema software realizzato secondo il paradigma ad oggetti. Tra tutti i tipi di diagrammi, in questa lezione, verrà utilizzato unicamente il diagramma delle classi (Class Diagram). Con i diagrammi delle classi viene descritta l'architettura di un sistema software in termini di classi e delle relazioni tra di esse. Il diagramma delle classi è una metodologia di progettazione a oggetti estremamente diffusa.

Poiché è nota sia la descrizione, sia la rappresentazione grafica di una classe, di seguito si riportano i tipi di legami (relazioni/associazioni) tra le classi:

- *associazione generica* - una classe ha un attributo il cui valore è una istanza di un'altra classe (o collezione di istanze di un'altra classe);
- *associazione aggregativa* (parte-tutto) - una classe ha un attributo il cui valore è una istanza di un'altra classe:
    - non c'è possesso esclusivo della parte che è sempre creata/distrutta indipendentemente dal tutto;
    - *associazione compositiva* (parte-tutto) - una classe ha un attributo il cui valore è una istanza di un'altra classe con possesso esclusivo della parte che è sempre creata/distrutta contestualmente al tutto.

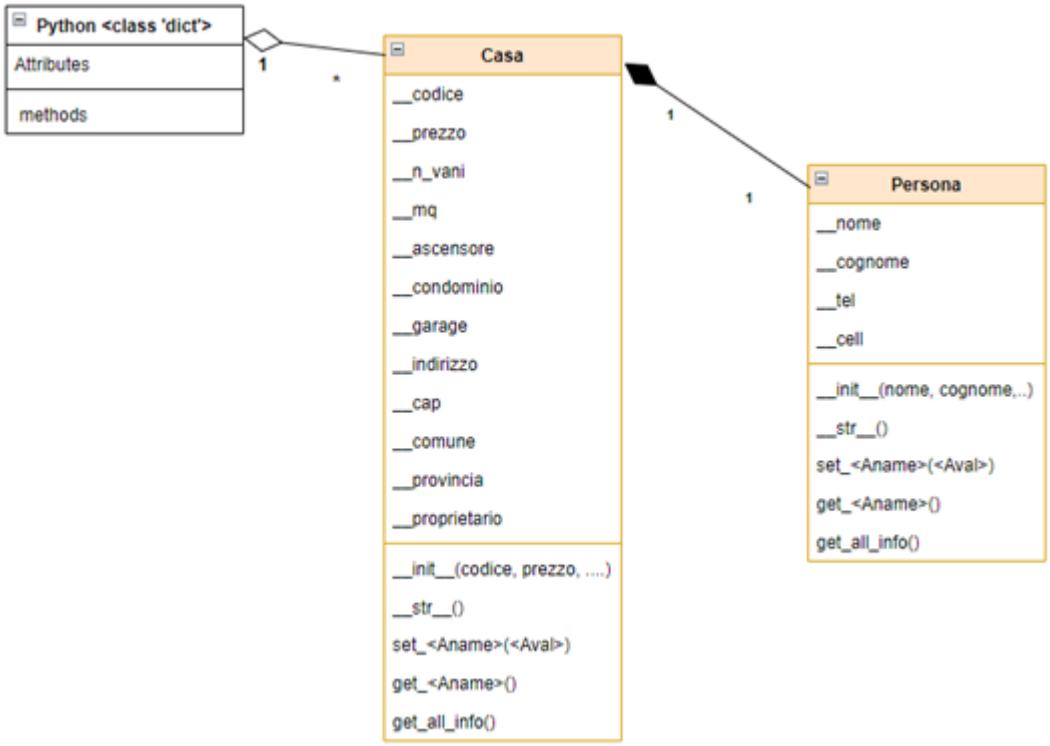
Di seguito viene illustrata la rappresentazione grafica delle diverse tipologie di *associazioni*.



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

A questo punto, sono noti tutti gli elementi per definire il layout del diagramma delle classi (UML) come mostrato nella figura seguente. In particolare, le associazioni descritte graficamente sono espresse dai seguenti attributi il cui valore è una istanza di un'altra classe:

- *__proprietario* presente nella classe ***Casa***;
- *value* presente nella classe '*dict'.*



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Formalizzazione della soluzione in Python

Completata la fase di progettazione, è possibile passare alla codifica delle classi in linguaggio Python cominciando dalla classe *Persona*.

Tutti gli attributi sono implementati ciascuno con una corrispondente variabile privata __nome, __cognome, __tel, __cell. Il metodo costruttore della classe assegna i corrispondenti valori alle variabili così come ricevuti attraverso i parametri. Inoltre, non sono necessari trattamenti interni dei valori ricevuti.

Ciascuno dei metodi setter assegna il valore ricevuto come parametro alla variabile corrispondente, mentre ciascuno dei metodi getter ritorna semplicemente il valore contenuto nella variabile corrispondente.

Il metodo *__str__()* viene riscritto per formattare opportunamente la visualizzazione a video di un oggetto di tipo Persona mediante la funzione *print()*. L'algoritmo concatena in un'unica stringa tutti gli attributi, premettendo a ciascuno il carattere «\n» e una descrizione del valore.

Il metodo *get_all_info()* realizza la formattazione in una stringa di tutti gli attributi per la scrittura su file di testo in formato CSV. In particolare, l'algoritmo costruisce una stringa che contiene i valori di tutti gli attributi separati con il carattere «,».

Di seguito, si riporta il codice Python della classe Persona.

```
class Persona:
    def __init__(self, nome, cognome, tel, cell):
        self.__nome = nome
        self.__cognome = cognome
        self.__tel = tel
        self.__cell = cell

    def set_nome(self, nome):
        self.__nome = nome

    def set_congome(self, cognome):
        self.__cognome = cognome

    def set_tel(self, tel):
        self.__tel = tel

    def set_cell(self, residenza):
        self.__cell = cel

    def get_nome(self):
        return self.__nome
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    def get_cognome(self):
        return self.__cognome

    def get_tel(self):
        return self.__tel

    def get_cell(self):
        return self.__cell
    def __str__(self):
        return "Nome: " + self.__nome + "\nCognome: " + self.__cognome +\
               "\nTel: " + self.__tel + "\nCell: " + self.__cell + "\n"

    def get_all_info(self):
        return self.__nome + "," + self.__cognome + "," + self.__tel + \
               "," + self.__cell
```

Per quanto riguarda la codifica della classe ***Casa***, tale classe possiede un attributo (*proprietario*) il cui valore è una istanza della classe *Persona* pertanto nella prima riga del file viene importata la definizione della classe riferita.

Gli attributi sono implementati ciascuno con una variabile privata come ad esempio __codice, __prezzo, __n_vani, ecc. Il metodo costruttore assegna il corrispondente valore ricevuto nel parametro senza alcun trattamento aggiuntivo. Ognuno dei metodi setter assegna il valore ricevuto come parametro alla variabile corrispondente, mentre ognuno dei metodi getter ritorna il valore della variabile corrispondente.

Il metodo *__str__()* viene riscritto per formattare opportunamente la visualizzazione a video di un oggetto di tipo Casa mediante la funzione *print().* In particolare, l'algoritmo concatena in un'unica stringa il valore di ciascun attributo, premettendo il carattere «\n» e una descrizione. Per ottenere la stringa formattata del proprietario, viene invocato il metodo *get_all_info()* sull'oggetto proprietario. Il metodo *get_all_info()* realizza la formattazione in una stringa di tutti gli attributi per la scrittura su file di testo in formato CSV. In particolare, l'algoritmo di quest'ultimo metodo costruisce una stringa che contiene i valori di tutti gli attributi separati con il carattere «,».

Di seguito, si riporta il codice Python della classe Casa.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import Persona

class Casa:

    def __init__(self, codice, prezzo, n_vani, mq, ascensore,\
                 condominio, garage, indi, cap, comune, pv, prop):
        self.__codice = codice
        self.__prezzo = prezzo
        self.__n_vani = n_vani
        self.__mq = mq
        self.__ascensore = ascensore
        self.__condominio = condominio
        self.__garage = garage
        self.__indirizzo = indi
        self.__cap = cap
        self.__comune = comune
        self.__provincia = pv
        self.__proprietario = prop

    def set_prezzo(self, prezzo):
        self.__prezzo=prezzo

    def set_n_vani(self,n_vani):
        self.__n_vani=n_vani

    def set_mq(self, mq):
        self.__mq= mq

    def set_ascensore(self, ascensore):
        self.__ascensore= ascensore

    def set_condominio(self, condominio):
        self.__condominio= condominio

    def set_garage(self, garage):
        self.__garage= garage


    def get_codice(self):
        return self.__codice

    def get_prezzo(self):
        return self.__prezzo

    def get_n_vani(self):
        return self.__n_vani

    def get_mq(self):
        return self.__mq

    def get_ascensore(self):
        return self.__ascensore

    def get_condominio(self):
        return self.__condominio

    def get_garage(self):
        return self.__garage

    def __str__(self):
        return "Codice: "+self.__codice+"\n\tPrezzo: "+str(format(self.__prezzo,".2f"))+\
               " €\n\tN. vani: "+str(self.__n_vani)+"\n\tMetri quadri: "+str(self.__mq)+\
               "\n\tAscensore: "+self.__ascensore+"\n\tCondominio: "+self.__condominio+\
               "\n\tGarage: "+self.__garage+"\n\tIndirizzo: "+self.__indirizzo+\
               "\n\tCAP: "+self.__cap+"\n\tComune: "+self.__comune+"\n\tProvincia: "+self.__provincia+\
               "\n\t-PROPRIETARIO-\n\t"+self.__proprietario.get_all_info()

    def get_all_info(self):
        return self.__codice+","+str(format(self.__prezzo,".2f"))+","+str(self.__n_vani)+\
               ","+str(self.__mq)+","+self.__ascensore+","+self.__condominio+","+self.__garage+\
               ","+self.__indirizzo+","+self.__cap+","+self.__comune+","+self.__provincia+\
               ","+self.__proprietario.get_all_info()
```

Si descrivono ora le due funzioni per la scrittura/lettura su file delle schede informative delle case: la funzione *salva_case(d)* e la funzione *carica_case(d)*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione *salva_case(d)* non restituisce nessun valore, riceve come parametro l'oggetto di tipo «*dizionario*» e scrive su file di testo in formato CSV tutte le schede delle case. L'algoritmo utilizzato dalla funzione *salva_case(d)* è strutturato nei seguenti passi:

- apre in scrittura il file (il nome del file viene mantenuto in una costante globale «*NOME_FILE*»);
- crea una lista di tutte le chiavi del dizionario passato come parametro;
- itera su tutte le chiavi:
    - chiama il metodo *get_all_info()* per formattare una riga di testo CSV;
    - scrive la riga nel file;
- chiude il file.

  Di seguito si riporta il codice Python della funzione *salva_case(d).*

```
def salva_case(d):
    csv_file = open(NOME_FILE, 'w')
    tutte_le_chiavi = d.keys()
    for chiave in tutte_le_chiavi:
        riga = d[chiave].get_all_info()
        csv_file.write(riga+'\n')

    csv_file.close()
```

La funzione *carica_case(d)* non restituisce nessun valore e riceve come parametro un oggetto di tipo «dizionario» in cui carica da file di testo in formato CSV tutte le schede delle case in gestione. L'algoritmo utilizzato è strutturato nei seguenti passi:

- apre in lettura il file (il nome del file viene mantenuto in una costante globale «*NOME_FILE*»);
- itera su tutte le righe del file:
    - legge la riga corrente;
    - crea, il metodo *split*(','), una lista con i dati contenuti nella riga di testo CSV appena letta;
    - crea un oggetto *Persona* con i dati del proprietario;
    - crea un oggetto *Casa* con i dati della casa e l'oggetto *Persona;*
    - aggiunge l'oggetto *Casa* nel dizionario con chiave il codice;
- chiude il file e stampa a video il numero di schede caricate.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta il codice Python della funzione *carica_case(d)*.

```
def carica_case(d):
    try:
        csv_file = open(NOME_FILE, 'r')
        for riga in csv_file:
            line_info = riga.rstrip('\n').split(',')
            p = Persona.Persona(line_info[11], line_info[12],\
                                line_info[13], line_info[14])
            c = Casa.Casa(line_info[0], float(line_info[1]),\
                          int(line_info[2]), float(line_info[3]),\
                          line_info[4], line_info[5], line_info[6],\
                          line_info[7],line_info[8],line_info[9],\
                          line_info[10], p)
            d[line_info[0]] = c
        csv_file.close()
        print("\t>>> " + str(len(d)) + " CASE in gestione <<<")

    except FileNotFoundError:
        print("\t>>>Nessuna casa in gestione<<<")

    except Exception as errore:
        print("\tERRORE: " + errore)
```

Infine, si descrivono le funzioni di supporto al *main()* per la simulazione del sistema. La funzione *display_menu()* visualizza a schermo le opzioni (con descrizione) del menu delle operazioni attivabili. Essa viene realizzata con una sequenza di chiamate della funzione *print()* come mostrato nel codice Python che segue.

```
def display_menu():
    print()
    print("MENU")
    print("1) Inserire una nuova casa da gestire")
    print("2) Eliminare una casa in gestione")
    print("3) Cerca case per prezzo")
    print("4) Stampare a schermo tutte le case in gestione")
    print("5) Terminare il programma")
    print()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione *chiedi_numero(prompt, tipo),* di cui si riporta il codice Python di seguito, viene utilizzata per richiedere l'input di un valore numerico riceve due parametri: il primo è il messaggio di invito da visualizzare, mentre il secondo è la tipologia numerica da controllare. In particolare, sono previste solo due tipologie «*int*» e «*float*». Ovviamente, la funzione restituisce il valore immesso dall'utente ed opportunamente controllato e verificato. L'algoritmo implementato itera la richiesta finché non viene digitato un valore corretto; il controllo viene effettuato catturando l'eventuale eccezione prodotta dal casting della stringa digitata dall'utente.

```
def chiedi_numero(prompt, tipo):
    valido = False
    while not valido:
        try:
            if tipo == "float":
                n = float(input(prompt))
            else:
                n = int(input(prompt))
            valido = True
        except:
            print("\tERRORE! Il valore inserito non è valido")
    return n
```

La funzione *chiedi_s_n(prompt)* è la funzione utilizzata per richiedere l'input di una stringa con valori sì o no (S/N). Essa riceve come unico parametro il messaggio di invito da visualizzare e restituisce il valore immesso dall'utente. L'algoritmo implementato itera la richiesta finché non viene digitato un valore tra i due previsti (S o N). Per semplicità di gestione, la stringa di input viene prima formattata con il metodo *upper()* e poi controllata. Di seguito si riporta il codice Python della funzione.

```
def chiedi_s_n(prompt):
    valido = False
    while not valido:
        s = (input(prompt)).upper()
        if s in ['S', 'N']:
            valido = True
    return s
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione *chiedi_cap()* -è la funzione utilizzata per richiedere l'input di un valore di tipo codice postale di cinque cifre. Essa restituisce il valore immesso dall'utente. Nell'algoritmo implementato, il controllo del CAP è stato semplificato verificando solo la forma sintattica (5 cifre decimali). Non è stata volutamente controllata la correttezza semantica del valore. Nel caso di CAP errato (numero di cifre errato), la richiesta viene reiterata finché non viene digitato un valore corretto. Per verificare che la stringa sia formata solo da cifre decimali, viene utilizzata la variabile *cifre*, inizializzata alla stringa formata da tutte le cifre decimali "0123456789". I caratteri che formano la stringa digitata vengono verificati uno alla volta mediante un ciclo iterativo while. Di seguito si riporta il codice Python della funzione.

```
def chiedi_cap():
    cifre_validate = 0
    cifre = "0123456789"
    while cifre_validate != 5:
        cifre_validate = 0
        s = input("Il CAP della casa è [cinque cifre]? ")
        if len(s) == 5:
            for c in s:
                if c in cifre:
                    cifre_validate +=1
        if len(s) != 5 or cifre_validate != 5:
            print("\tERRORE! Il valore inserito non è valido")
    return s
```

La funzione *chiedi_sigla_pv()*, invece, è la funzione utilizzata per richiedere l'input di un valore stringa corrispondente ad una sigla di provincia. Essa restituisce il valore immesso dall'utente. L'algoritmo implementato itera la richiesta finché non viene digitato una sigla corretta (la correttezza viene verificata cercando il valore immesso nella lista delle sigle di provincia contenute in *"sigle_pv*"). Di seguito si riporta il codice Python della funzione.

```
def chiedi_sigla_pv():
    valida = False
    while not valida:
        s_pv = (input("Il codice della provincia è? ")).upper()
        if s_pv in sigle_pv:
            valida = True
        else:
            print("\tERRORE! La sigla inserita non è valida")
    return s_pv
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione *chiedi_min_max()* è la funzione utilizzata per richiedere l'input dei valori di prezzo minimo e massimo per effettuare la ricerca delle case di possibile interesse. Essa restituisce i valori di minimo e massimo immessi dall'utente. Nell'algoritmo implementato si fa uso della funzione *chiedi_numero(prompt, tipo).* I valori digitati dall'utente vengono riordinati (facendo uso delle funzioni *min()* e *max()*) per evitare che inavvertitamente l'utente abbia scambiato il minimo con il massimo nel digitare i valori. Di seguito si riporta il codice Python della funzione.

```
def chiedi_min_max():
    prezzo_min = chiedi_numero("Inserire il prezzo minimo: ", "float")
    prezzo_max = chiedi_numero("Inserire il prezzo MASSIMO: ", "float")
    p_min = min(prezzo_min, prezzo_max)
    p_max = max(prezzo_min, prezzo_max)
    return p_min, p_max
```

La funzione *chiedi_info_casa(cod)* è la funzione utilizzata per richiedere in input tutti i dati del proprietario e della casa. Essa riceve come parametro il codice di una nuova casa e restituisce un oggetto di tipo Casa oppure un codice di errore. Nell'algoritmo implementato si fa uso delle funzioni di supporto definite per richiedere i dati necessari. Vengono prima richiesti i dati del proprietario con i quali viene creato un oggetto Persona. Successivamente vengono poi richiesti tutti i dati della casa e viene creato, quindi, un oggetto casa con codice quello passato come parametro. Di seguito si riporta il codice Python della funzione.

```
def chiedi_info_casa(cod):
    try:
        #Informazioni del proprietario: dati tipo stringa generica
        nome = input("Nome del proprietario? ")
        cognome = input("Cognome del proprietario? ")
        t = input("Il numero di telefono fisso? ")
        c = input("Il numero di cellulare? ")
        p = Persona.Persona(nome, cognome, t, c)
        #Richiesta dati di tipo numerico
        prezzo = chiedi_numero("Inserire il prezzo della casa: ", "float")
        n_vani = chiedi_numero("Inserire il numero dei vani della casa: ", "int")
        mq = chiedi_numero("Inserire i metri quadri della casa: ", "float")

        #Richiesta dati di tipo [S/N]
        ascensore = chiedi_s_n("La casa dispone di ascensore [S/N]? ")
        condominio = chiedi_s_n("La casa è inserita in un condominio [S/N]? ")
        garage = chiedi_s_n("La casa dispone di un garage [S/N]? ")
        #Richiesta del CAP
        cap = chiedi_cap()

        #Richiesta dati tipo stringa generica
        indirizzo = input("Indirizzo della casa? ")
        comune = input("Il nome del comune è? ")
        pv = chiedi_sigla_pv()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
            c = Casa.Casa(cod, prezzo, n_vani, mq, ascensore, condominio, garage,\
                          indirizzo, cap, comune, pv, p)
            return c
        except Exception as err:
            print("ERRORE: ",err)
            return 1
```

Le costanti e le tabelle globali utilizzate per migliorare la leggibilità e per semplificare gli algoritmi implementati sono le seguenti:

```
#Costanti globali utilizzate
NOME_FILE = "case_in_gestione.csv"
AGGIUNGI_CASA = 1
CANCELLA_CASA = 2
CERCA_CASE = 3
CASE_IN_GESTIONE = 4
QUIT_CHOICE = 5
#Tabella/lista delle sigle di provincia
sigle_pv = [
    "AG","AL","AN","AO","AQ","AR","AP","AT","AV","BA",\
    "BT","BL","BN","BG","BI","BO","BZ","BS","BR","CA",\
    "CL","CB","CI","CE","CT","CZ","CH","CO","CS","CR",\
    "KR","CN","EN","FM","FE","FI","FG","FC","FR","GE",\
    "GO","GR","IM","IS","SP","LT","LE","LC","LI","LO",\
    "LU","MC","MN","MS","MT","VS","ME","MI","MO","MB",\
    "NA","NO","NU","OG","OT","OR","PD","PA","PR","PV",\
    "PG","PU","PE","PC","PI","PT","PN","PZ","PO","RG",\
    "RA","RC","RE","RI","RN","Roma","RO","SA","SS","SV",\
    "SI","SR","SO","TA","TE","TR","TO","TP","TN","TV",\
    "TS","UD","VA","VE","VB","VC","VR","VV","VI","VT"]
```

Infine, la funzione *main()* attiva l'esecuzione del programma e permette la simulazione delle operazioni realizzate. Le parti salienti dell'algoritmo codificato nella funzione *main()* sono:

- la creazione e il popolamento iniziale del dizionario delle case in gestione. Le schede vengono lette dal file di testo in formato CSV;
    - l'esecuzione di un ciclo iterativo finché l'utente non sceglie di terminare il programma. Ad ogni iterazione vengono eseguite le seguenti azioni:
        - visualizzazione del menu;
        - richiesta all'utente di effettuare una scelta;
        - esecuzione della funzione scelta;
        - visualizzazione del risultato;
    - prima di terminare l'esecuzione del programma, solo se il dizionario non è vuoto e sono state apportate delle modifiche il dizionario viene salvato su file.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta il codice Python della funzione *main()*.

```
def main():
    agenzia = dict()
    carica_case(agenzia)
    modifiche = False
    choice = 0
    while choice != QUIT_CHOICE:
        display_menu()

        choice = int(input('Inserire la scelta: '))

        if choice == AGGIUNGI_CASA:
            codice = (input("Inserire il codice univoco della casa: ")).upper()
            if codice not in agenzia:
                c = chiedi_info_casa(codice)
                if c!=1:
                    agenzia[codice] = c
                    modifiche = True
            else:
                print("CODICE casa già  presente!")

        elif choice == CANCELLA_CASA:
            codice = input(("Inserire il codice della casa da eliminare: ")).upper()
            print()
            if codice in agenzia:
                del agenzia[codice]
                print("La casa con codice "+codice+" è stata cancellata!")
                modifiche = True
            else:
                print("Codice univoco della casa non corretto")
            print()
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Simulazione di alcune operazioni

Si propongono ora le simulazioni di alcune operazioni effettuate con il programma sviluppato. La prima simulazione è l'avvio del programma e il caricamento di due schede contenute nel file "*case_in_gestione.csv*". Come evidenziato nel risultato dell'elaborazione seguente, viene scelta l'opzione di stampa a video delle schede.

```
        >>> 2 CASE in gestione <<<

MENU
1) Inserire una nuova casa da gestire
2) Eliminare una casa in gestione
3) Cerca case per prezzo
4) Stampare a schermo tutte le case in gestione
5) Terminare il programma

Inserire la scelta: 4

Codice: S001
        Prezzo: 156000.00 €
        N. vani: 6
        Metri quadri: 145.0
        Ascensore: N
        Condominio: N
        Garage: S
        Indirizzo: Via Dante Alighieri 34
        CAP: 50100
        Comune: Capannori
        Provincia: LU
        -PROPRIETARIO-
        Alberto,Rossi,0583987654,338934567

Codice: C001
        Prezzo: 220000.00 €
        N. vani: 8
        Metri quadri: 160.0
        Ascensore: S
        Condominio: S
        Garage: N
        Indirizzo: Via G. Arcangeli 16
        CAP: 56124
        Comune: Pisa
        Provincia: PI
        -PROPRIETARIO-
        Pasquale,Bianchi,050765843,337012378
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La seconda, invece, è la simulazione di una operazione di inserimento una nuova scheda:

```
MENU
1) Inserire una nuova casa da gestire
2) Eliminare una casa in gestione
3) Cerca case per prezzo
4) Stampare a schermo tutte le case in gestione
5) Terminare il programma

Inserire la scelta: 1
Inserire il codice univoco della casa: V001
Nome del proprietario? Giuseppe
Cognome del proprietario? Giallino
Il numero di telefono fisso? 050435782
Il numero di cellulare? 334567891
Inserire il prezzo della casa: 300000
Inserire il numero dei vani della casa: 10
Inserire i metri quadri della casa: 190
La casa dispone di ascensore [S/N]? n
La casa e' inserita in un condominio [S/N]? n
La casa dispone di un garage [S/N]? s
Il CAP della casa è [cinque cifre]? 56126
Indirizzo della casa? Via Grazia Deledda 124
Il nome del comune è? Pontedera
Il codice della provincia è? pi
```

La terza è la simulazione di una richiesta di operazione di stampa a video delle schede presenti in archivio:

```
MENU
1) Inserire una nuova casa da gestire
2) Eliminare una casa in gestione
3) Cerca case per prezzo
4) Stampare a schermo tutte le case in gestione
5) Terminare il programma

Inserire la scelta: 4

Codice: S001
        Prezzo: 156000.00 €
        N. vani: 6
        Metri quadri: 145.0
        Ascensore: N
        Condominio: N
        Garage: S
        Indirizzo: Via Dante Alighieri 34
        CAP: 50100
        Comune: Capannori
        Provincia: LU
        -PROPRIETARIO-
        Alberto,Rossi,0583987654,338934567
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
Codice: C001
        Prezzo: 220000.00 €
        N. vani: 8
        Metri quadri: 160.0
        Ascensore: S
        Condominio: S
        Garage: N
        Indirizzo: Via G. Arcangeli 16
        CAP: 56124
        Comune: Pisa
        Provincia: PI
        -PROPRIETARIO-
        Pasquale,Bianchi,050765843,337012378

Codice: V001
        Prezzo: 300000.00 €
        N. vani: 10
        Metri quadri: 190.0
        Ascensore: N
        Condominio: N
        Garage: S
        Indirizzo: Via Grazia Deledda 124
        CAP: 56126
        Comune: Pontedera
        Provincia: PI
        -PROPRIETARIO-
        Giuseppe,Giallino,050435782,334567891
```

La quarta è la simulazione di una ricerca delle case con prezzo compreso tra un prezzo minimo ed un prezzo massimo:

```
MENU
1) Inserire una nuova casa da gestire
2) Eliminare una casa in gestione
3) Cerca case per prezzo
4) Stampare a schermo tutte le case in gestione
5) Terminare il programma

Inserire la scelta: 3

Inserire il prezzo minimo: 150000
Inserire il prezzo MASSIMO: 200000
Codice: S001
        Prezzo: 156000.00 €
        N. vani: 6
        Metri quadri: 145.0
        Ascensore: N
        Condominio: N
        Garage: S
        Indirizzo: Via Dante Alighieri 34
        CAP: 50100
        Comune: Capannori
        Provincia: LU
        -PROPRIETARIO-
        Alberto,Rossi,0583987654,338934567
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La quinta è la simulazione di una operazione di eliminazione di una casa:

```
        >>> 3 CASE in gestione <<<

MENU
1) Inserire una nuova casa da gestire
2) Eliminare una casa in gestione
3) Cerca case per prezzo
4) Stampare a schermo tutte le case in gestione
5) Terminare il programma

Inserire la scelta: 2
Inserire il codice univoco della casa da eliminare: c001

La casa con codice C001 è stata cancellata!
```

La sesta e ultima simulazione riguarda l'operazione di chiusura del programma con salvataggio delle schede su file:

```
MENU
1) Inserire una nuova casa da gestire
2) Eliminare una casa in gestione
3) Cerca case per prezzo
4) Stampare a schermo tutte le case in gestione
5) Terminare il programma

Inserire la scelta: 5
Salvataggio delle case in gestione....
Programma TERMINATO!
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).